Data Example: Phosphorous Levels

Phosphorous Example A – Exploratory Data Analysis

Mahedi Hasan and Jun Zhu

1 Phosphorous Example A: Exploratory Data Analysis

In this implementation, we will be using phosphorous example A from lecture to explore the data and spatial correlation.

1.1 Phosphorous A

Inorganic P (ppm)
(explanatory variable)

0.7	1.2	3.6	19.0
1.0	0.8	18.5	27.8
1.4	0.9	21.9	20.9

Organic P (ppm)
(response variable)

0.1	1.9	5.1	26.5
1.8	2.0	26.3	38.6
2.3	1.3	30.1	28.4

In the phosphorous A example, a corn field is divided into a grid of plots, with 3 rows and 4 columns, where each plot is indexed from 1 to 12. In each of the plots, the organic phosphorous level of a randomly selected corn plant is measured in ppm, as well as the inorganic phosphorous level in the soil. We want to quantify the relationship between organic phosphorous and inorganic phosphorous.

First, we load four R packages we will need: `spdep`, `spatialreg`, `ggplot2`, and `MASS`.

```
library(spdep)
library(spatialreg)
library(ggplot2)
library(MASS)
```

Next, we load the data into R by row. We create two data vectors, and call them `phos.inorganic` and `phos.organic`.

```
phos.inorganic <- c(0.7, 1.2, 3.6, 19.0,
                    1.0, 0.8, 18.5, 27.8,
                    1.4, 0.9, 21.9, 20.9)
# inorganic phosphorous: explanatory variable

phos.organic <- c(0.1, 1.9, 5.1, 26.5,
                  1.8, 2.0, 26.3, 38.6,
                  2.3, 1.3, 30.1, 28.4)
# organic phosphorous: response
```

We also create a vector for indexing the 12 plots.

```
plot <- as.factor(seq(1, 12))  # indicator for plot ID
```

To check that the data vectors of inorganic and organic phosphorous are loaded correctly, we can display them as in Figure 1 by using the `matrix()` function and applying it to the two data vectors. We specify the number of columns to be 4 and `byrow=TRUE`.

```
matrix(phos.inorganic, ncol = 4, byrow = TRUE)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  0.7  1.2  3.6 19.0
## [2,]  1.0  0.8 18.5 27.8
## [3,]  1.4  0.9 21.9 20.9
```

```
matrix(phos.organic, ncol = 4, byrow = TRUE)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  0.1  1.9  5.1 26.5
## [2,]  1.8  2.0 26.3 38.6
## [3,]  2.3  1.3 30.1 28.4
```

Having checked the data vectors, we go ahead and create a data frame called `phosA` in order to keep all the data vectors in one place. We use the function `cbind.data.frame()` to combine the data vectors into a single data frame, which will consist of the data vectors of inorganic phosphorous, organic phosphorous, and plot indices.

```
# create a data frame
phosA <- cbind.data.frame(phos.inorganic, phos.organic, plot)
```

Having created a data frame, we next explore summary statistics of the data. We can apply the `summary()` function to the data frame, `phosA`.

```
# descriptive stats
summary(phosA)
```

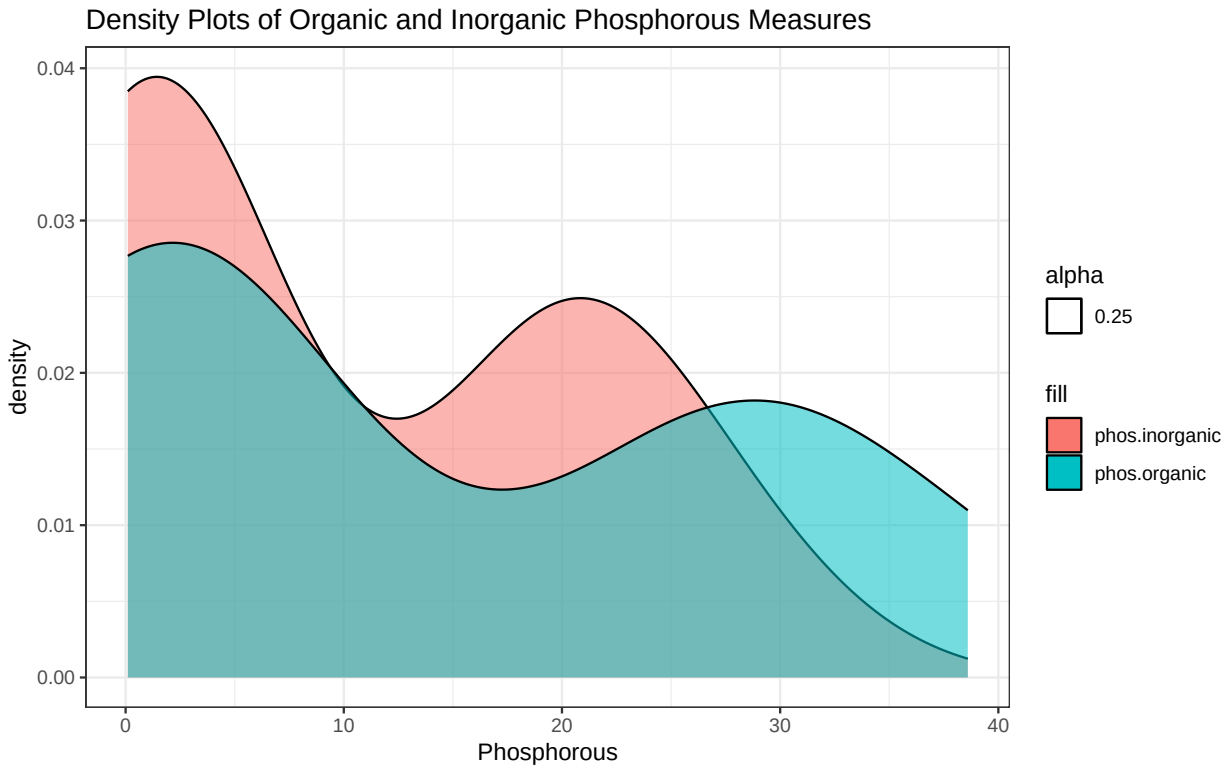
```
##  phos.inorganic    phos.organic         plot
##  Min.   : 0.700   Min.   : 0.100   1      :1
##  1st Qu.: 0.975   1st Qu.: 1.875   2      :1
##  Median : 2.500   Median : 3.700   3      :1
##  Mean   : 9.808   Mean   :13.700   4      :1
##  3rd Qu.:19.475   3rd Qu.:26.975   5      :1
##  Max.   :27.800   Max.   :38.600   6      :1
##                                    (Other):6
```

We see from the descriptive statistics that while the organic phosphorous has a lower minimum than the inorganic phosphorous, it is generally greater than the inorganic phosphorous according to the other quantiles.

We want to visualize the data in order to gain insights about the data and the relationship between organic and inorganic phosphorous. We can first create density plots, overlaying organic and inorganic phosphorous. We will use the `ggplot()` functions and syntax from the `ggplot2` package.

To create the density plot, we will use the `geom_density()` function from the `ggplot2` package.

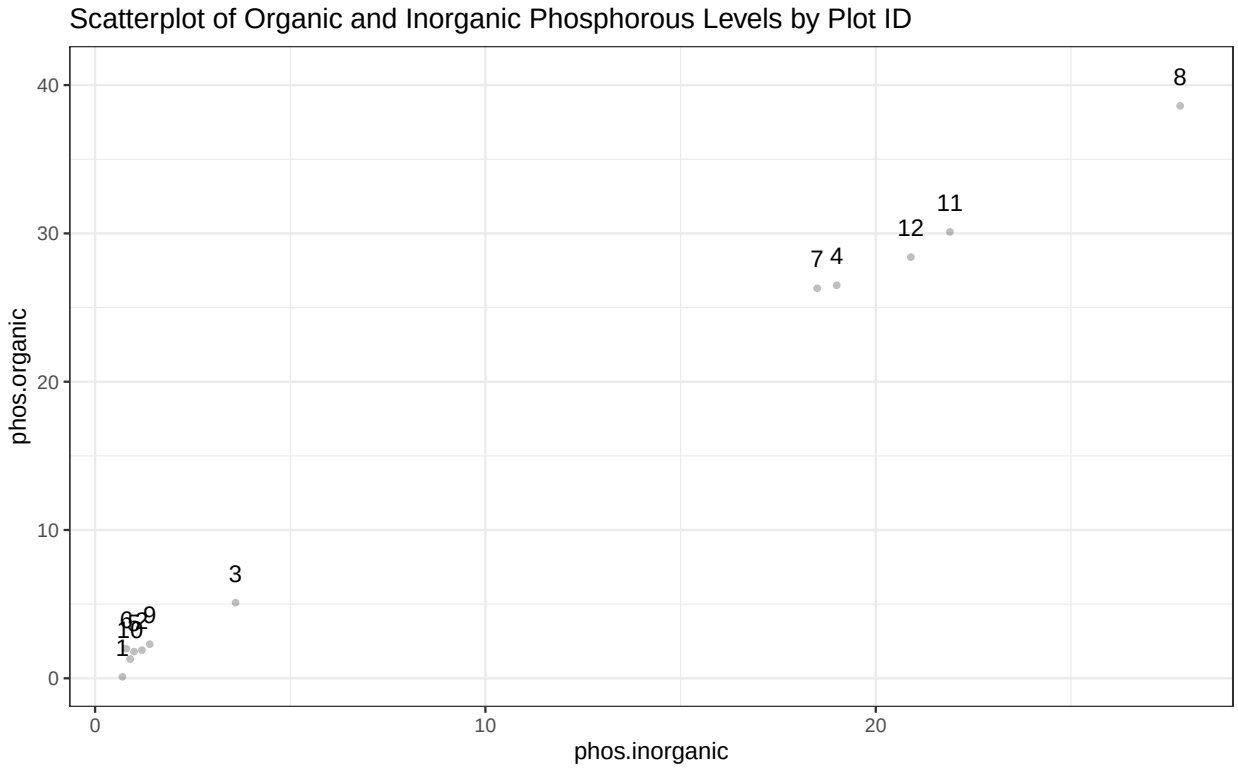
```
ggplot(phosA) +
  geom_density(aes(x = phos.inorganic, fill = "phos.inorganic", alpha = 0.25)) +
  geom_density(aes(x = phos.organic, fill = "phos.organic", alpha = 0.25)) +
  ggtitle("Density Plots of Organic and Inorganic Phosphorous Measures") +
  xlab("Phosphorous") +
  theme_bw()
```



From the density plots, we see that there is greater density of inorganic phosphorous at lower and middle levels that drop off at higher phosphorous levels, whereas organic phosphorous has more higher-level phosphorous at the upper tail and lower density at the lower tail.

We can also look at the relationship between inorganic and organic phosphorous by plots using scatter plots. In the scatter plots, we can substitute the plot indices for points so that we can identify the plots with their respective data points. We will use the `geom_point()` function and also the additional `geom_text()` function to overlay the plot indexes over the points, both from the `ggplot2` package.

```
ggplot(phosA, aes(phos.inorganic, phos.organic)) +
  geom_point(size = 1, alpha = 0.25) +
  geom_text(label = rownames(phosA), size = 4, position = position_nudge(x = 0, y = 2)) +
  ggtitle("Scatterplot of Organic and Inorganic Phosphorous Levels by Plot ID") +
  theme_bw()
```



From this first scatter plot, we can see that there is a positive association between organic and inorganic phosphorous. There are not any phosphorous observations between about 10 and 15 in both the organic and inorganic measurements. In addition, plots 7, 4, 12, 11, and 8 are in the upper quadrant of phosphorous for both organic and inorganic.

2 Spatial Correlation

2.1 Creating Neighbors

In preparation for the spatial analysis, we define the neighbors using the R function `cell2nb()` from the `spdep` package. To create first-order neighbors that share edges, we specify the argument `type="rook"` in the `cell2nb()` function as follows.

```
# Generate neighborhood structure object
phos.nb <- cell2nb(nrow = 3, ncol = 4, type = "rook")
```

Applying the `str()` function to `phos.nb`, we can explore the structure of the neighborhood structure object.

```
str(phos.nb)
```

```
## List of 12
##  $ : int [1:2] 2 5
##  $ : int [1:3] 1 3 6
##  $ : int [1:3] 2 4 7
##  $ : int [1:2] 3 8
##  $ : int [1:3] 1 6 9
```

```
##  $ : int [1:4] 2 5 7 10
##  $ : int [1:4] 3 6 8 11
##  $ : int [1:3] 4 7 12
##  $ : int [1:2] 5 10
##  $ : int [1:3] 6 9 11
##  $ : int [1:3] 7 10 12
##  $ : int [1:2] 8 11
##  - attr(*, "class")= chr "nb"
##  - attr(*, "call")= language cell2nb(nrow = 3, ncol = 4, type = "rook")
##  - attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##  - attr(*, "cell")= logi TRUE
##  - attr(*, "rook")= logi TRUE
##  - attr(*, "sym")= logi TRUE
```

We see that the output is a list of 12 items. The first item corresponds to the first plot. It tells us that for plot 1, its first-order neighbors are plots 2 and 5.

We then create a list of spatial weights using the R function `nb2listw()` from the `spdep` package. Here, we specify `style=B` for binary spatial weights.

```
# Create list of spatial weights for neighborhood structure object
phos.lw <- nb2listw(phos.nb, style = "B")
```

To explore the structure of the list of spatial weights, we apply the `str()` function to the list of spatial weights, `phos.lw`.

```
str(phos.lw)
```

```
## List of 3
##  $ style     : chr "B"
##  $ neighbours:List of 12
##   ..$ : int [1:2] 2 5
##   ..$ : int [1:3] 1 3 6
##   ..$ : int [1:3] 2 4 7
##   ..$ : int [1:2] 3 8
##   ..$ : int [1:3] 1 6 9
##   ..$ : int [1:4] 2 5 7 10
##   ..$ : int [1:4] 3 6 8 11
##   ..$ : int [1:3] 4 7 12
##   ..$ : int [1:2] 5 10
##   ..$ : int [1:3] 6 9 11
##   ..$ : int [1:3] 7 10 12
##   ..$ : int [1:2] 8 11
##   ..- attr(*, "class")= chr "nb"
##   ..- attr(*, "call")= language cell2nb(nrow = 3, ncol = 4, type = "rook")
##   ..- attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##   ..- attr(*, "cell")= logi TRUE
##   ..- attr(*, "rook")= logi TRUE
##   ..- attr(*, "sym")= logi TRUE
##  $ weights   :List of 12
```

```
##   ..$ : num [1:2] 1 1
##   ..$ : num [1:3] 1 1 1
##   ..$ : num [1:3] 1 1 1
##   ..$ : num [1:2] 1 1
##   ..$ : num [1:3] 1 1 1
##   ..$ : num [1:4] 1 1 1 1
##   ..$ : num [1:4] 1 1 1 1
##   ..$ : num [1:3] 1 1 1
##   ..$ : num [1:2] 1 1
##   ..$ : num [1:3] 1 1 1
##   ..$ : num [1:3] 1 1 1
##   ..$ : num [1:2] 1 1
##   ..- attr(*, "mode")= chr "binary"
##   ..- attr(*, "B")= logi TRUE
##  - attr(*, "class")= chr [1:2] "listw" "nb"
##  - attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##  - attr(*, "call")= language nb2listw(neighbours = phos.nb, style = "B")
##  - attr(*, "zero.policy")= logi FALSE
```

The structure above gives a list of 3 items: style, neighbors, and weights. Style reminds us the spatial weights we specified. In this case, we had specified binary weights given by the argument `style=B`. The second item is the neighborhood structure object, `phos.nb`. Lastly, we have the weights. These are the binary spatial weights we specified. For example, the first entry tells us that for plot 1, the weights applied to plots 2 and 5 (plot 1's neighbor plots) are both 1. All other plots that are not neighbors have zero weight, which are not shown.

3 Testing for Spatial Correlation using Moran's I

Now that we've created the neighbors and list of spatial weights, we can quantify spatial patterns. Specifically, we will start with testing for spatial patterns in the phosphorous data using Moran's I.

We perform Moran's I test on organic phosphorous data using the `moran.test()` function and the list of spatial weights, `phos.lw`.

```
# Test spatial correlation of organic phosphorous
set.seed(1)
moran.test(phos.organic, phos.lw, alternative = "two.sided")
```

```
##
##  Moran I test under randomisation
##
## data:  phos.organic
## weights: phos.lw
##
## Moran I statistic standard deviate = 3.1104, p-value = 0.001869
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##        0.59123455       -0.09090909        0.04809856
```

For the two-sided Moran's I test above, the observed Moran's I statistic is `0.59123455`. The p-value is `0.001869`. We reject the null hypothesis at the 5% level. There is strong evidence against the null hypothesis of complete spatial randomness in the organic phosphorous levels.

Similarly, we test for spatial patterns in inorganic phosphorous using Moran's I.

```
set.seed(2)
moran.test(phos.inorganic, phos.lw, alternative = "two.sided")
```

```
##
##  Moran I test under randomisation
##
## data:  phos.inorganic
## weights: phos.lw
##
## Moran I statistic standard deviate = 3.1316, p-value = 0.001739
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##        0.59602011       -0.09090909        0.04811754
```

For the two-sided Moran's I test above, the observed Moran's I statistic value is `0.59602011`. The p-value is `0.001739`. We again reject the null hypothesis at the 5% level. There is strong evidence against the null hypothesis of complete spatial randomness in the inorganic phosphorous levels.